

Lab 1 — Image Sharpening

Objective

To write a C++ application that sharpens an image in the PPM (P6) format, and to study how the time taken by each of the five phases — file read, S1, S2, S3 and file write — varies with the size of the input image.

Approach

Blurring an image removes its fine detail, so if the blurred version is subtracted from the original, what remains is only the detail. Adding that detail back on top of the original makes it stronger and the image looks sharper. This is done in three steps:

- **S1 — Smoothing.** Every pixel is replaced by the average of the 3×3 block of pixels around it, which blurs the image.
- **S2 — Finding the details.** The smoothed image is subtracted from the original, pixel by pixel. What is left is the detail that the blurring removed.
- **S3 — Sharpening.** The details are added back to the original, so the detail is counted twice and the edges stand out more.

Corner and edge pixels. A pixel at a corner or along an edge does not have all 8 neighbours, so the 3×3 block would fall outside the image. Only the neighbours that actually exist are used, and the sum is divided by how many pixels were actually added — 4 at a corner, 6 along an edge and 9 inside the image.

Values outside 0 to 255. Each colour value is stored in one byte, so it can only hold 0 to 255, but the subtraction in S2 can give a negative value and the addition in S3 can give a value up to 510. Such a value would wrap around if stored directly, so each result is calculated in a larger integer type and then brought back into range: anything below 0 is made 0 and anything above 255 is made 255. This is called clamping.

Measurement method

The time was measured with the C++ `chrono` library, using `steady_clock` sampled before and after each of the five phases. Every image was run **5 times and the average taken**, with one extra un-timed warm-up run before them so that the file-read figure is not distorted by a cold page cache. All times are in **microseconds (µs)**.

Table 1 — The 5 individual runs for one image

Image 4 (1280 × 910), shown to illustrate how the average is obtained.

Run	File read	S1	S2	S3	File write	Total
1	249 159	105 206	84 020	82 497	199 663	720 545
2	237 399	141 667	111 918	97 444	184 176	772 604
3	253 294	153 152	114 718	92 627	153 137	766 928
4	232 320	121 043	87 373	77 133	142 473	660 342
5	249 052	130 384	89 078	100 288	216 120	784 922
Average	244 244.8	130 290.4	97 421.4	89 997.8	179 113.8	741 068.2

The same procedure was followed for all seven images.

Table 2 — Average time for each phase, all images

Image	Dimensions	Pixels	File read	S1	S2	S3	File write	Total
1	450 × 180	81 000	20 237.4	10 396.2	5 911.0	5 751.4	11 775.6	54 071.6
2	786 × 393	308 898	66 606.6	35 703.6	24 240.0	21 683.0	47 479.4	195 712.6
3	1024 × 575	588 800	113 989.2	63 317.6	51 320.2	40 205.2	90 265.4	359 097.6
4	1280 × 910	1 164 800	244 244.8	130 290.4	97 421.4	89 997.8	179 113.8	741 068.2
5	1500 × 1000	1 500 000	299 188.6	160 241.2	129 049.6	99 707.2	232 868.2	921 054.8
6	2271 × 1500	3 406 500	648 665.6	343 033.8	254 551.0	231 469.2	495 422.0	1 973 141.6
7	3750 × 2470	9 262 500	1 664 876.8	889 225.4	644 067.8	596 375.4	1 324 148.6	5 118 694.0

Observations

1. **Every phase grows in proportion to the number of pixels.** Image 7 has about 114 times as many pixels as image 1 and takes roughly 95 times as long, so the time taken per pixel stays nearly the same however large the image is.
2. **File read is the slowest of the five phases,** and reading and writing the file together take a little more than half of the total time — more than all three transformations put together. This is because the PPM file is read and written one byte at a time.
3. **S1 is the slowest of the three transformations,** taking roughly one and a half times as long as S2 or S3. This is because S1 reads nine neighbouring pixels for every output pixel, while S2 and S3 read only one.
4. **The same image does not take the same time on every run.** For image 4 the slowest of the five runs took about a fifth longer than the fastest, because other processes on the machine and the operating system's scheduling affect each run. This is why the experiment is repeated five times and averaged instead of being measured once.